Regressive Plasticity Schedule: A Two-Stage Post-Training Schedule for ARC Program Synthesis

Jason Feng

https://x.com/iamjasonfeng

June 5, 2026

Abstract

This paper reports an experimental study of Regressive Plasticity Schedule (RPS), a two-stage post-training schedule: first train on easier data at a higher learning rate, then train on harder data at a substantially lower learning rate. Earlier work has studied easy-to-hard curricula combined with learning-rate reductions; the contribution here is an evaluation of this scheduling principle in LLM post-training for ARC program synthesis, not the introduction of the general principle. I tested RPS on Qwen3-8B using Alibaba Model Studio managed DPO fine-tuning with LoRA. The control condition, Equal Plasticity Schedule (EPS), used the same model, two-stage datasets, within-stage cosine scheduler, and Stage 1 checkpoint, but did not reduce the Stage 2 base learning rate. On ARC-AGI-1 public evaluation, exact test-output accuracy was 17/419 for RPS versus 10/419 for EPS. On ARC-AGI-2 public evaluation, neither condition solved any test outputs, but 234/240 attempted programs executed without error for RPS, compared with 188/240 for EPS. These preliminary results support further investigation of lower Stage 2 learning rates under this curriculum. They do not establish broad reasoning improvements or isolate the interaction between curriculum ordering and learning-rate reduction.

Update notice (November 6, 2026): This revision adds discussion and citations of prior work that combined easy-to-hard curricula with learning-rate reductions. I was unaware of these papers when I independently developed RPS and conducted the original experiments. The paper now distinguishes this prior scheduling principle from the experimental study of RPS in LLM post-training reported here. This revision also adds a citation for Codex's assistance with writing this paper. This paper was originally published on June 5, 2026.

1. Introduction

ARC-AGI is designed to test abstraction and generalization from small numbers of examples.[1][2] ARC tasks are difficult for ordinary language-model prompting because success requires inferring a latent transformation rule and applying it exactly to new grids. Recent ARC systems increasingly combine learning, search, program synthesis, test-time adaptation, and synthetic data rather than relying on a single model call.[4]

This paper explores a small post-training hypothesis: for ARC-style reasoning, it may be useful to reduce model plasticity when moving from easier foundational tasks to harder target tasks. The motivation is to help a model acquire reusable patterns and then adapt to harder tasks without overwriting those patterns too aggressively. Human development supplied the original intuition for this experiment, but that analogy is a motivation rather than an established explanation of the results. Broader reasoning improvement remains a goal for future evaluation.

In this experiment, plasticity is operationalized through the learning rate. Stage 1 trains on easier ARC-style examples with a higher learning rate. Stage 2 trains on harder ARC-AGI-2 examples with 10% of the Stage 1 learning rate. This general scheduling principle has direct precedents in empirical curriculum-learning work and theoretical work on jointly optimizing curricula and learning rates.[9][10] The contribution of this paper is

the reported Qwen3-8B DPO/LoRA experiment and its ARC accuracy and program-reliability measurements. It does not claim to originate the principle or establish its first application to LLMs. The controlled comparison asks whether lowering the Stage 2 base learning rate improves performance under the fixed two-stage curriculum used here.

2. Prior Work

2.1 ARC-AGI and Program Synthesis

ARC-AGI tasks are grid transformation problems where each task provides a small set of training input-output pairs and one or more test inputs.[3] A system must infer the transformation and produce exact test outputs. Program synthesis is a natural approach: generate a candidate transformation program, execute it on the training examples, reject programs that fail the demonstrations, and apply surviving programs to test inputs.

The 2025 ARC Prize results showed that strong ARC systems can use refinement loops, synthetic data, test-time training, and specialized models.[4] This paper studies whether RPS can improve the reliability of the kind of program-synthesis behavior that may be useful inside such systems. Program synthesis is treated as a concrete reasoning behavior that can be measured, not as the final scope of RPS.

2.2 Curriculum Learning and Learning-Rate Schedules

Curriculum learning trains models on data ordered by difficulty. Learning-rate schedules vary the optimizer step size during training. Combining easier examples at a higher learning rate with harder examples at a lower learning rate predates this experiment.

Sarao Mannelli et al. studied the interaction between curricula and overparameterisation, using theoretical models and image-classification experiments.[9] Their Fashion-MNIST experiment trained on easier examples first, then harder examples with the learning rate reduced to one third of its initial value. This is a direct empirical precedent for the broad RPS recipe, not merely a general connection to curriculum learning. Their random-order comparison also used the learning-rate reduction, unlike the EPS comparison here, which holds the curriculum fixed and changes the Stage 2 base learning rate.

Mignacco and Mori's original July 2025 preprint developed a statistical-physics framework for jointly optimizing example difficulty and learning rate in an idealized teacher-student learning problem.[10] Section 3.1 and Figure 3 describe an optimized easy-to-hard curriculum with learning-rate annealing and a sharp drop at the easy-to-hard transition. Thus, coupling a learning-rate drop to the curriculum boundary is also an explicit theoretical precedent. Their analysis does not establish that the same schedule is optimal for LLM post-training.

The present experiment differs in model, training objective, and evaluation: it studies Qwen3-8B with DPO and LoRA on ARC program synthesis rather than the classification settings above. Its contribution is evidence from that setting, not a new general scheduling principle. Both RPS and EPS used cosine schedules within stages; the comparison isolates the Stage 2 base learning rate under the chosen curriculum, not the necessity or optimality of coupling the two.

2.3 Preference Optimization

The experiment used DPO-style preference fine-tuning.[6] Each training example contained a prompt, a chosen response, and a rejected response. The chosen response contained an ARC reasoning trace, a Python transformation program, and predicted test outputs. The rejected response was generated by a weaker Qwen3-0.6B model under normal solving conditions, not by instructing the model to intentionally be wrong.

3. Method

3.1 Regressive Plasticity Schedule

RPS is a two-stage schedule:

```
Stage 1: easier ARC-style examples, higher learning rate
Stage 2: harder ARC-AGI-2 examples, lower learning rate
```

In this experiment:

```
Stage 1 learning rate: 1e-5
Stage 2 learning rate: 1e-6
```

Stage 2 therefore used 10% of the Stage 1 learning rate.

3.2 Equal Plasticity Schedule Control

EPS is the control condition. It used the same two-stage structure and the same Stage 1 model, but did not reduce the Stage 2 learning rate:

```
Stage 1 learning rate: 1e-5
Stage 2 learning rate: 1e-5
```

Both RPS and EPS used cosine learning-rate schedules within each stage. EPS therefore controls for the fact that each stage already has within-stage learning-rate decay. The main difference between RPS and EPS is the stage-level reduction in the Stage 2 base learning rate.

3.3 Base Model and Fine-Tuning Setup

The base model was Qwen3-8B.[5] Fine-tuning was performed with Alibaba Model Studio managed DPO fine-tuning using LoRA.[6][7]

Shared settings:

```
Base model: qwen3-8b
Fine-tuning method: DPO
Adapter method: LoRA
Validation split: 1%
Maximum sequence length: 32k
Scheduler: cosine
RPO alpha: 1
Stage 1 dataset size: 400 preference pairs
Stage 2 dataset size: 600 preference pairs
Replay: none
ARC-AGI public evaluation data used for training: none
```

3.4 Dataset

The ARC DPO dataset contained 1,000 preference pairs.

Stage 1 contained 400 pairs from easier ARC-style training tasks. It contained 400 unique tasks. Most examples came from ARC-AGI-1 or ARC-AGI-1/2-overlap tasks, with a small number of ARC-AGI-2 training examples used to fill the quota.

Stage 2 contained 600 pairs from ARC-AGI-2 training tasks only. It contained 250 unique tasks, with some duplicate task IDs allowed to fill the 600-pair quota. No task IDs overlapped between Stage 1 and Stage 2.

Each DPO pair contained:

```
prompt: ARC training examples and test inputs
chosen: reasoning, Python transform program, predicted test outputs
rejected: weaker model response generated under normal solving conditions
```

The chosen responses came from Trelis/arc-agi-2-reasoning-5, a dataset of ARC reasoning traces and Python program-synthesis solutions.[8] Rejected responses were generated by Qwen3-0.6B. Both chosen and rejected responses were normalized into the same labeled structure:

```
<think>
...
</think>

Python program:
def transform(grid):
    ...

Predicted test outputs:
...
```

The rejected responses were not instructed to be wrong. They were generated by querying the weaker model as if it were solving the task normally. Because Qwen3-0.6B is weak on ARC, these responses were expected to be lower quality than the chosen responses.

4. Evaluation

4.1 ARC-AGI-1 Public Evaluation

ARC-AGI-1 public evaluation was run over 400 tasks.[3] Each task used three attempts, yielding 1,200 generation attempts. The official score was computed over 419 test outputs. A test output was counted correct if any attempt exactly matched the ground truth output.

4.2 ARC-AGI-2 Public Evaluation

ARC-AGI-2 public evaluation was run over 120 tasks with two attempts per test input grid.[3] The official score was computed over 167 test outputs.

4.3 Program-Synthesis Reliability

The models were evaluated not only by final exact output accuracy but also by whether they produced executable Python programs. An attempt was counted as an executed-program output when the generated Python program ran without error and produced valid grid outputs that were used for scoring. Parsed JSON fallbacks were not counted as program-synthesis success.

This metric is not a substitute for task accuracy. It also is not a full measure of general reasoning. It measures whether the model stayed in the intended program-synthesis mode and produced usable executable artifacts, which I use as one observable proxy for structured reasoning behavior.

5. Results

5.1 ARC-AGI-1 Exact Test-Output Accuracy

RPS improved ARC-AGI-1 public evaluation score relative to EPS.

Model	Correct test outputs	Total test outputs	Score
RPS	17	419	4.06%
EPS	10	419	2.39%

At the task level, RPS solved 14/400 tasks, compared with 10/400 for EPS.

5.2 ARC-AGI-1 Program-Synthesis Reliability

RPS also improved program-synthesis reliability on ARC-AGI-1.

Metric	RPS	EPS
Attempts	1,200	1,200
Program executions without error	1,145	870
Attempts scored from executed programs	1,086	812
Invalid attempts	26	250
Tasks with all attempts scored from executed programs	306/400	137/400
Tasks with at least one executed-program attempt	398/400	382/400
Tasks with zero executed-program attempts	2/400	18/400

5.3 ARC-AGI-2 Exact Test-Output Accuracy

Neither model solved any ARC-AGI-2 public-evaluation test outputs.

Model	Correct test outputs	Total test outputs	Score
RPS	0	167	0.00%
EPS	0	167	0.00%

This is the central limitation of the current result. RPS did not solve ARC-AGI-2 in this standalone setup.

5.4 ARC-AGI-2 Program-Synthesis Reliability

Despite the zero exact score on ARC-AGI-2, RPS substantially improved program-synthesis reliability.

Metric	RPS	EPS
Attempts	240	240
Program executions without error	234	188
Attempts scored from executed programs	214	176
Parsed-JSON fallback attempts	18	19
Invalid attempts	8	45
Tasks with both attempts scored from executed programs	100/120	60/120
Tasks with at least one executed-program attempt	114/120	116/120
Tasks with zero executed-program attempts	6/120	4/120

Paired attempt-level comparison:

Comparison	Count
Both RPS and EPS produced executed-program outputs	160/240
RPS only produced an executed-program output	54/240
EPS only produced an executed-program output	16/240
Neither produced an executed-program output	10/240

The paired difference in ARC-AGI-2 executed-program attempts was significant under an exact McNemar test, with p = 5.85e-6. This statistical test should be interpreted cautiously because attempts are not fully independent, but it supports the view that the observed reliability difference is unlikely to be random noise.

5.5 Token and Thinking-Budget Checks

Thinking-budget saturation was rare and does not appear to explain the RPS/EPS difference.

ARC-AGI-1:

```
RPS thinking-budget hits: 0/1200
EPS thinking-budget hits: 15/1200
```

ARC-AGI-2:

```
RPS thinking-budget hits: 1/240
EPS thinking-budget hits: 6/240
```

For ARC-AGI-1, excluding test outputs with thinking-budget-hit attempts produced:

Model	Correct test outputs	Kept test outputs	Score
RPS	17	405	4.20%
EPS	10	405	2.47%

Thus the ARC-AGI-1 result remains favorable to RPS after excluding thinking-budget-hit cases.

6. Discussion

6.1 What RPS Appears To Improve

RPS produced two encouraging signals. First, ARC-AGI-1 exact test-output accuracy was higher than for EPS. This measures successful task solutions, not merely formatting or program syntax. However, the small number of solved outputs and the limited experimental coverage do not establish a general reasoning improvement.

Second, on both ARC-AGI-1 and ARC-AGI-2, RPS improved program-synthesis reliability. RPS made the model more likely to produce a runnable Python transformation program and less likely to produce invalid responses.

This matters because ARC systems often need to generate, test, repair, and rank candidate programs. A model that reliably produces syntactically and semantically executable transformation programs may be more useful inside a larger ARC system than a model that produces free-form reasoning or invalid outputs.

An open question is whether the same schedule helps in other reasoning domains. The results motivate that question, but they do not measure retention of foundational skills or representational drift. Those possible mechanisms require separate tests. The prior curriculum-learning results provide context for the scheduling hypothesis; they do not independently validate its effectiveness in this LLM setting.

6.2 Why EPS Is The Key Control

Both RPS and EPS used two stages, cosine schedules within each stage, the same Stage 1 checkpoint, and the same Stage 2 dataset. EPS kept the Stage 2 base learning rate equal to Stage 1, while RPS reduced it to 10% of Stage 1. EPS therefore tests the effect of the lower Stage 2 base learning rate under this particular curriculum.

The difference is not explained by the mere presence of within-stage cosine decay, which both conditions share. However, this comparison cannot determine whether the curriculum and learning-rate reduction interact beneficially, whether lowering the learning rate without this curriculum would work equally well, or whether another decay schedule would perform better. Answering those questions requires additional curriculum-ordering and learning-rate controls. EPS also does not distinguish the general recipe from its prior empirical and theoretical precedents.

6.3 Limitations

This experiment has several important limitations.

First, RPS did not solve any ARC-AGI-2 public-evaluation test outputs in the standalone setup. The ARC-AGI-2 result is therefore a reliability result, not an accuracy result.

Second, the experiment used one base model and one dataset construction. The result should be tested across model sizes, preference datasets, SFT/RL variants, multiple random seeds, and non-ARC reasoning benchmarks. This is especially important because the long-term motivation for RPS is general reasoning, not only ARC program synthesis.

Third, the rejected responses were generated by a weak model and were not independently verified as incorrect. The assumption was that the chosen program-synthesis traces were higher quality than Qwen3-0.6B generated responses. This is plausible but imperfect.

Fourth, some Stage 2 task IDs were duplicated to fill the 600-pair quota. Future experiments should test larger and cleaner datasets with less duplication.

Fifth, the current results are public-evaluation results and should not be treated as private leaderboard performance.

Sixth, the study does not compare a sufficient range of curriculum orderings and learning-rate schedules to establish that the boundary-coupled drop is necessary or optimal. Its contribution is limited to the reported LLM experiment with an existing scheduling principle.

7. Conclusion

This paper evaluated a simple curriculum-coupled learning-rate schedule in Qwen3-8B DPO/LoRA post-training for ARC program synthesis. The general principle of moving from easier examples at a higher learning rate to harder examples at a lower learning rate has prior empirical and theoretical support.[9][10] The contribution here is the reported experiment: RPS achieved higher ARC-AGI-1 exact test-output accuracy than EPS and improved program-synthesis reliability on both public evaluations.

The result is preliminary. Neither condition solved ARC-AGI-2 test outputs, and broad cross-domain reasoning improvement remains untested. The comparison supports further study of lower Stage 2 learning rates under this curriculum, but does not establish a curriculum-learning-rate interaction. Further work should include multiple seeds, matched alternative curricula and learning-rate schedules, evaluation inside larger ARC solvers, and reasoning benchmarks beyond ARC.

Acknowledgments

I independently developed the RPS experiment and ran the experiments myself. I used Codex to help with the training workflow, dataset preparation, evaluation notebooks, analysis, and writing of this paper.[11]

References

[1] Chollet, F. (2019). On the Measure of Intelligence. arXiv:1911.01547.

[2] ARC Prize Foundation. ARC Prize 2026.

[3] ARC Prize Foundation. ARC Prize Guide.

[4] ARC Prize Foundation. ARC Prize 2025 Results and Analysis.

[5] Qwen Team. (2025). Qwen3 Technical Report. arXiv:2505.09388.

[6] Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290.

[7] Hu, E. J. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

[8] Trelis Research. arc-agi-2-reasoning-5.

[9] Sarao Mannelli, S., Ivashynka, Y., Saxe, A. M., and Saglietti, L. (2024). Tilting the Odds at the Lottery: the Interplay of Overparameterisation and Curricula in Neural Networks. Proceedings of the 41st International Conference on Machine Learning, PMLR 235, 34586-34602.

[10] Mignacco, F., and Mori, F. (2025). A statistical physics framework for optimal learning. arXiv:2507.07907, original version submitted July 10, 2025. See Section 3.1 and Figure 3.

[11] OpenAI. Codex.